INSTITUTO FEDERAL DO ESPÍRITO SANTO
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

**PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS**

# GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS

ALEGRE
2026

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

# GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS

Trabalho de Conclusão de Curso apresentado à Coordenadoria do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Espírito Santo, como requisito parcial para a obtenção do título de Analista em Desenvolvimento de Sistemas.

Orientador: Prof. D.Sc Carlos Alexandre Siqueira da Silva

ALEGRE

2026

Dados Internacionais de Catalogação-na-Publicação (CIP)
(Biblioteca Nilo Peçanha do Instituto Federal do Espírito Santo)

X999y

Gestão Operacional e Controle de Lanchonetes: Desenvolvimento de um Sistema Informatizado para Automação de Processos/ PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS. – 2026.

46 f: il; 30 cm.

Orientador: Carlos Alexandre  Siqueira da Silva

Trabalho de Graduação – Instituto Federal do Espírito Santo, Curso de Análise e Desenvolvimento de Sistemas, 2026.

1. Redes neurais (Computação). 2. Expressão facial – Processamento de dados. 3. Sistemas de reconhecimento de padrões. 4. Processamento de imagens – Técnicas digitais. 5. Percepção de padrões. 6. Engenharia Elétrica. I. XXXXXXXX, XXXXXXXXXX XXXXXXXXXX. II. Instituto Federal do Espírito Santo. III. Título.

CDD 000.00

Elaborada por XXXXXXXXXXXXXXXXXXXXX – CRB-X/ES - XXX

**PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS**

# GESTÃO OPERACIONAL E CONTROLE DE LANCHONETES: DESENVOLVIMENTO DE UM SISTEMA INFORMATIZADO PARA AUTOMAÇÃO DE PROCESSOS

Trabalho de Conclusão de Curso apresentado à Coordenadoria do Curso de Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia do Espírito Santo, como requisito parcial para a obtenção do título de Analista em Desenvolvimento de Sistemas.

Aprovado em 01 de Agosto de 2026

**COMISSÃO EXAMINADORA**

**Prof. D.Sc Carlos Alexandre Siqueira da Silva**
Instituto Federal do Espírito Santo
Orientador

**Profa. Dra. Fulana de Tal**
Instituto Federal do Espírito Santo
Examinadora

**Prof. Dr. Cicrano de Tal**
Instituto Federal do Espírito Santo
Examinador

A dedicatória é um elemento opcional. Contém o oferecimento do trabalho adeterminada pessoa ou a pessoas (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b).

## DECLARAÇÃO DO AUTOR

Eu, **PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS**, declaro que este trabalho foi desenvolvido com base em meus próprios estudos e que não houve uso de material de terceiros, exceto o que foi citado.

Declaro para fins de pesquisa acadêmica, didática e técnico-científica, que este Trabalho de Conclusão de Curso pode ser parcialmente utilizado, desde que seja feita a devida referência à fonte e ao autor.

Alegre, 22 de abril de 2026

PEDRO HENRIQUE RIGUETE DE OLIVEIRA MARTINS

## AGRADECIMENTOS

test elemento opcional. Localiza-se após a folha de aprovação e deve ser dirigido àqueles que realmente contribuíram, de maneira relevante,para a elaboração do trabalho. Deve-se utilizar uma linguagem simples (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b).

**Epigrafe:** É um elemento opcional. É uma citação relacionada ao assunto do trabalho desenvolvido, seguida da indicação de autoria (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2011b). Deve-se seguir as regras do uso da citação NBR 10.520/2002.

## RESUMO

Este trabalho apresenta o desenvolvimento de uma aplicação web destinada ao apoio da gestão operacional de uma lanchonete. A proposta parte da identificação de dificuldades associadas ao uso de registros manuais, como erros no lançamento de pedidos, demora no atendimento, inconsistências no controle de produtos e limitação do acesso a informações gerenciais. O sistema foi estruturado com o framework Laravel e a linguagem PHP, utilizando MySQL para a persistência dos dados e Redis como mecanismo de cache. A arquitetura adota o padrão Model–View–Controller e o controle de acesso baseado em funções, com o objetivo de separar responsabilidades e restringir as funcionalidades conforme o perfil do usuário. Entre as funções previstas estão o cadastro e a autenticação de usuários, o gerenciamento de produtos, a apresentação do catálogo, o carrinho de compras e o acompanhamento dos pedidos. O uso de Docker contribui para a padronização do ambiente de execução, enquanto Git e GitHub apoiam o controle de versões. Espera-se que a solução reduza a ocorrência de falhas operacionais, centralize as informações do estabelecimento e proporcione maior agilidade ao atendimento e ao acompanhamento das atividades.



Palavras-chave: Gestão operacional. Sistemas web. Lanchonetes. Laravel. Automação de processos.

**ABSTRACT**

This work presents the development of a web application designed to support the operational management of a snack bar. The proposal is based on difficulties associated with manual records, including order-entry errors, service delays, inconsistencies in product control, and limited access to management information. The system was structured with the Laravel framework and PHP, using MySQL for data persistence and Redis as a caching mechanism. Its architecture follows the Model–View–Controller pattern and role-based access control to separate responsibilities and restrict features according to each user profile. The planned features include user registration and authentication, product management, catalog display, a shopping cart, and order tracking. Docker contributes to a standardized execution environment, while Git and GitHub support version control. The solution is expected to reduce operational failures, centralize business information, and improve the speed of customer service and activity monitoring.

Keywords: Operational management. Web systems. Snack bars. Laravel. Process automation.

## LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxo de consulta ao Redis no sistema Flashfood . . . . . 28
Figura 2 – Modelo físico do banco de dados do sistema Flashfood (visão geral) 34
Figura 3 – Detalhe do modelo físico do banco de dados – região superior esquerda 35
Figura 4 – Detalhe do modelo físico do banco de dados – região superior direita 36
Figura 5 – Detalhe do modelo físico do banco de dados – região inferior esquerda 37
Figura 6 – Detalhe do modelo físico do banco de dados – região inferior direita 38

## LISTA DE TABELAS

Tabela 1 – Comparativo entre trabalhos correlatos e sistema proposto . . . . . 20
Tabela 2 – Requisitos funcionais do sistema . . . . . 31
Tabela 3 – Requisitos não funcionais do sistema . . . . . 32
Tabela 4 – Testes de cadastro, autenticação e controle de acesso . . . . . 40
Tabela 5 – Testes de produtos, cardápio, carrinho e pedidos . . . . . 41

# LISTA DE QUADROS

## LISTA DE SIGLAS

| | |
|---|---|
| Ifes | Instituto Federal do Espírito Santo |
| MVC | Model–View–Controller |
| PHP | PHP: Hypertext Preprocessor |
| RBAC | Role-Based Access Control (Controle de Acesso Baseado em Funções) |
| RNF | Requisitos Não Funcionais |
| RF | Requisitos Funcionais |
| SQL | Structured Query Language (Linguagem de Consulta Estruturada) |
| SGBD | Sistema de Gerenciamento de Banco de Dados |

**SUMÁRIO**


**1 INTRODUÇÃO . . . . . 15**
**2 JUSTIFICATIVA . . . . . 16**
**3 REFERENCIAL TEÓRICO . . . . . 18**
**3.0.1 Trabalhos correlatos . . . . . 18**
3.0.1.1 Sistema de Point of Sale em cafeteria . . . . . 18
3.0.1.2 Restaurant Management System . . . . . 18
3.0.1.3 POS web com Laravel para Meetup Station . . . . . 19
3.0.1.4 POS multipagamento para UMKM Mr. Boba . . . . . 19
3.0.1.5 Sistema de caixa web para Natta Coffee . . . . . 19
**3.0.2 Comparação entre trabalhos correlatos e sistema proposto . . . 20**
3.1 MYSQL . . . . . 20
3.2 PHP . . . . . 21
3.3 LARAVEL . . . . . 21
3.4 GIT . . . . . 21
3.5 GITHUB . . . . . 22
3.6 BLADE (SISTEMA DE TEMPLATES) . . . . . 22
3.7 VISUAL STUDIO CODE . . . . . 22
3.8 CONTROLE DE ACESSO BASEADO EM FUNÇÕES (RBAC) . . . . 23
3.9 DOCKER . . . . . 23
**3.9.1 Testes automatizados de software . . . . . 24**
**3.9.2 Testes de ponta a ponta (E2E) . . . . . 25**
**3.9.3 Cypress . . . . . 25**
**3.9.4 Uso do Cypress no sistema FlashFood . . . . . 26**
3.10 REDIS . . . . . 26
**3.10.1 Cache em Aplicações Web . . . . . 27**
**3.10.2 Redis como Camada de Cache . . . . . 27**
**3.10.3 Escalabilidade Horizontal . . . . . 28**
**3.10.4 Aplicação do Redis no Sistema Desenvolvido . . . . . 28**
**4 REQUISITOS DE SOFTWARE . . . . . 30**
4.1 REQUISITOS FUNCIONAIS . . . . . 30
4.2 REQUISITOS NÃO FUNCIONAIS . . . . . 30

**5 MODELAGEM DE SOFTWARE . . . . . 33**
**6 TESTES E VALIDAÇÃO . . . . . 39**
6.1 OBJETIVO E ESCOPO DOS TESTES . . . . . 39
6.2 ARQUIVOS E CENÁRIOS DE TESTE . . . . . 39
**6.2.1 Cadastro, autenticação e controle de acesso . . . . . 39**
**6.2.2 Produtos, cardápio, carrinho e pedidos . . . . . 39**
6.3 AMBIENTE E EXECUÇÃO . . . . . 39
6.4 RESULTADOS . . . . . 39
6.5 EVIDÊNCIAS . . . . . 42
**7 CONCLUSÃO . . . . . 43**
7.1 TRABALHOS FUTUROS . . . . . 43
**REFERÊNCIAS . . . . . 45**

# 1 INTRODUÇÃO

Devido à convivência com o ambiente de trabalho de uma lanchonete, foram identificadas diversas necessidades passíveis de automação, uma vez que o estabelecimento não opera com sua eficiência máxima. Isso ocorre principalmente pela ausência de um sistema informatizado para controle de estoque, gerenciamento de vendas, controle de pedidos e emissão de comandas, entre outras funcionalidades essenciais.

A inexistência dessa ferramenta tem ocasionado erros operacionais, atrasos no atendimento e sobrecarga de trabalho para os funcionários. Esses fatores podem resultar na insatisfação dos clientes, perda de vendas e na criação de um ambiente de trabalho estressante e ineficiente, afetando negativamente proprietários, colaboradores e consumidores.

Diante desse cenário, surgiu a necessidade de desenvolver um sistema capaz de otimizar os processos internos e melhorar a eficiência operacional, sem a necessidade de aumento no quadro de funcionários. Além disso, busca-se elevar o volume de vendas por meio da melhoria na organização e execução das atividades da lanchonete.

Com o desenvolvimento do sistema, espera-se promover uma melhoria significativa no funcionamento do estabelecimento, tornando o ambiente mais organizado, eficiente e agradável. Ademais, objetiva-se aumentar a satisfação de clientes, funcionários e gestores, contribuindo para a fidelização dos clientes, incremento nas vendas e melhoria na qualidade dos serviços prestados, constituindo assim um diferencial competitivo no mercado.

## 2 JUSTIFICATIVA

A informatização de processos operacionais tem se tornado um fator relevante para a competitividade de pequenos negócios. Em lanchonetes, a agilidade no atendimento e a precisão das informações influenciam diretamente a satisfação dos clientes, o desempenho da equipe e a capacidade de controle do gestor. Quando pedidos, produtos e estoque são administrados sem integração, o estabelecimento fica mais exposto a perdas, atrasos, retrabalho e decisões baseadas em percepções incompletas.

Este trabalho se justifica, inicialmente, pela necessidade prática de desenvolver uma solução capaz de apoiar a rotina de uma lanchonete real ou representativa desse tipo de negócio. A proposta busca reduzir a dependência de registros manuais e centralizar informações importantes em um sistema web, permitindo que diferentes perfis de usuário acessem funcionalidades compatíveis com suas responsabilidades. Dessa forma, pretende-se tornar o atendimento mais organizado e oferecer ao gestor melhores condições para acompanhar a operação.

Do ponto de vista acadêmico, o projeto favorece a aplicação integrada de conhecimentos de análise e desenvolvimento de sistemas, banco de dados, engenharia de software, segurança da informação, arquitetura de software e documentação técnica. A construção de uma aplicação com Laravel e MySQL permite explorar conceitos como MVC, ORM, migrações, autenticação, autorização, regras de negócio e testes funcionais. Além disso, o uso de Docker e Git/GitHub aproxima o desenvolvimento das práticas adotadas em ambientes profissionais.

Sob a perspectiva tecnológica, a escolha do Laravel se justifica pela maturidade do ecossistema PHP e pela existência de recursos que reduzem a complexidade de tarefas recorrentes em aplicações web. O MySQL é adequado por ser amplamente utilizado, possuir documentação consolidada e atender às necessidades de armazenamento relacional do sistema. O Docker contribui para evitar diferenças entre ambientes de desenvolvimento, enquanto o Git permite registrar a evolução do código-fonte e apoiar a rastreabilidade das alterações.

Também há uma justificativa social e econômica. Pequenos empreendedores nem

sempre dispõem de recursos para adquirir sistemas comerciais robustos ou excessivamente complexos para sua realidade. Uma solução acadêmica, construída a partir de requisitos bem definidos, pode demonstrar caminhos viáveis para modernização de rotinas e melhoria da organização interna. Ainda que o sistema proposto não substitua integralmente plataformas comerciais completas, ele pode atender a necessidades centrais e servir como base para futuras evoluções.

Portanto, a relevância deste trabalho está em articular uma demanda operacional concreta com uma proposta técnica fundamentada. O estudo busca produzir um artefato de software útil, documentado e avaliável, contribuindo tanto para a formação acadêmica do autor quanto para a discussão sobre automação de processos em pequenos estabelecimentos alimentícios.

# 3 REFERENCIAL TEÓRICO

Nesta seção, são apresentados os principais conceitos e fundamentos teóricos utilizados no desenvolvimento deste trabalho, abordando os conhecimentos necessários para a compreensão das tecnologias e metodologias empregadas no sistema.

## 3.0.1 Trabalhos correlatos

Os trabalhos correlatos permitem observar como problemas semelhantes foram tratados em outros contextos e quais lacunas permanecem em relação à proposta deste pré-projeto.

### 3.0.1.1 Sistema de Point of Sale em cafeteria

Hidayat e Farell (2023) desenvolveram um sistema de informação POS para uma cafeteria, utilizando Laravel, PHP, MySQL e o padrão MVC. O estudo informa que registros manuais de relatórios e estoque podiam gerar demora, perda de documentos e erros de cadastro. A solução foi projetada com método waterfall e diagramas como atividade, contexto, caso de uso e fluxograma.

A proximidade com este trabalho está no uso de Laravel, MySQL e no foco em ambiente de venda de alimentos e bebidas. A diferença principal está na proposta deste pré-projeto de enfatizar também comandas, RBAC e organização operacional integrada.

### 3.0.1.2 Restaurant Management System

Jingga e Limantara (2014) propuseram um modelo de Restaurant Management System desenvolvido com PHP e MySQL. O sistema buscava apoiar a comunicação entre garçom, chef e caixa em tempo real, além de contemplar processos ligados a pedidos e pagamentos.

Esse trabalho é relevante por tratar diretamente de restaurantes e por reconhecer a importância da integração entre atendimento e preparação. Contudo, a proposta aqui apresentada adota Laravel, Docker e controle de acesso baseado em papéis, além de direcionar a solução para lanchonetes de pequeno porte.

#### 3.0.1.3 POS web com Laravel para Meetup Station

Khairunnisa, Arwani e Hanggara (2022) desenvolveram um sistema POS web para a UMKM Meetup Station, negócio do setor de alimentos e bebidas. Segundo o estudo, a empresa realizava registros de transações em livro físico e cálculos manuais, o que causava erros. A solução utilizou Laravel, PHP, PostgreSQL e metodologia waterfall.

Os testes indicaram validação funcional completa e funcionamento responsivo em diferentes navegadores e resoluções. A principal contribuição para este pré-projeto é a demonstração de viabilidade de uma solução web para pequenos negócios alimentícios. Como diferença, o sistema proposto neste trabalho utilizará MySQL e dará maior ênfase a estoque, comandas e RBAC.

#### 3.0.1.4 POS multipagamento para UMKM Mr. Boba

Lathief, Hasdiana e Lubis (2023) apresentaram um sistema POS multipagamento baseado em Laravel para a UMKM Mr. Boba. O estudo relata dificuldades com vendas manuais, ausência de alternativas de pagamento não presencial e demora no fechamento de relatórios. O desenvolvimento adotou o método RAD e testes black-box.

Esse trabalho contribui por evidenciar a importância de relatórios computadorizados e de maior agilidade no atendimento. A proposta deste pré-projeto, entretanto, não se concentra em múltiplos meios de pagamento, mas na gestão operacional integrada da lanchonete.

#### 3.0.1.5 Sistema de caixa web para Natta Coffee

Rusmara (2025) desenvolveu um sistema de caixa web para a Natta Coffee com framework Laravel, visando melhorar transações de venda, controle de estoque e relatórios financeiros. O estudo adotou RAD e indicou ganhos na precisão de registros, velocidade de pagamento e emissão de relatórios.

A relação com este trabalho é direta, pois ambos tratam de estabelecimentos de alimentação e de sistemas web com Laravel. A diferença está na ampliação proposta para controle de comandas, perfis de usuário e estrutura de permissões.

### 3.0.2 Comparação entre trabalhos correlatos e sistema proposto

O Quadro 1 sintetiza as principais características dos trabalhos analisados e as compara com o sistema proposto.

Tabela 1 – Comparativo entre trabalhos correlatos e sistema proposto

| **Trabalho** | **Domínio** | **Tecnologias** | **Funcionalidades centrais** | **Relação com o sistema proposto** |
|---|---|---|---|---|
| Hidayat e Farell (2023) | Cafeteria | Laravel, PHP, MySQL, MVC | POS, vendas, relatórios e estoque | Similar no domínio e nas tecnologias; menor ênfase em RBAC e comandas. |
| Jingga e Limantara (2014) | Restaurante | PHP, MySQL | Pedido, comunicação garçom-chef-caixa e pagamento | Similar na integração operacional; não utiliza Laravel nem Docker. |
| Khairunnisa, Arwani e Hanggara (2022) | Alimentos e bebidas | Laravel, PHP, PostgreSQL | POS, transações e relatórios | Similar em POS web; usa PostgreSQL e não prioriza RBAC. |
| Lathief, Hasdiana e Lubis (2023) | Bebidas/UMKM | Laravel | POS multipagamento e relatórios | Similar na automação de vendas; foco maior em pagamento. |
| Rusmara (2025) | Cafeteria | Laravel | Caixa web, estoque e relatórios financeiros | Próximo ao tema; proposta atual amplia perfis, comandas e controle operacional. |
| Sistema proposto | Lanchonete | Laravel, PHP, MySQL, Docker, Git/GitHub | Produtos, pedidos, comandas, estoque, usuários, RBAC e relatórios | Integra gestão operacional, controle de acesso e ambiente padronizado para pequeno negócio. |

Fonte: Fonte: Elaborado pelo autor (2026)..

A análise dos trabalhos correlatos indica que há soluções voltadas a POS, caixa e gestão de restaurantes, mas a proposta deste pré-projeto se diferencia pela combinação de gestão operacional de lanchonete, comandas, estoque, Laravel, MySQL, Docker, versionamento e controle de acesso baseado em papéis.

Essa integração busca atender a uma necessidade prática: organizar a rotina do estabelecimento sem impor complexidade excessiva ao usuário final.

## 3.1 MYSQL

De acordo com Milani (2007), o cenário de globalização e o avanço da automação, impulsionados pela popularização da internet, tornaram o armazenamento de dados um passo primordial para a migração de negócios para o ambiente digital. Nesse contexto, o MySQL surge para suprir essa demanda de organização de informações.

O MySQL é um Sistema Gerenciador de Banco de Dados (SGBD) relacional distribuído sob um modelo de licenciamento duplo, que inclui uma versão de código aberto. Embora tenha sido projetado inicialmente para aplicações de pequeno e médio porte, a ferramenta evoluiu para atender também a projetos de maior escala. Segundo Milani (2007), o MySQL apresenta robustez e recursos comparáveis aos de sistemas proprietários, como SQL Server e Oracle.

## 3.2 PHP

Conforme afirma Dall'Oglio (2015), o PHP — que originalmente significava Personal Home Page — é uma linguagem de programação criada por Rasmus Lerdorf no outono de 1994. Inicialmente, a ferramenta consistia em um conjunto de scripts escritos em linguagem C, destinados à criação de páginas dinâmicas para que o autor monitorasse o acesso ao seu currículo na internet. Com o aumento da popularidade entre outros usuários, Lerdorf adicionou novos recursos, como a interação com bancos de dados. Em 1995, o código-fonte foi liberado, permitindo a colaboração de novos desenvolvedores. Vale ressaltar que, durante um curto período, a linguagem também foi denominada FI (Forms Interpreter).

## 3.3 LARAVEL

De acordo com N.Ricardo (2025), o Laravel é um *framework* para o desenvolvimento de aplicações web em PHP. A ferramenta oferece recursos para tarefas recorrentes, como roteamento, acesso a bancos de dados, autenticação e organização da aplicação, podendo ser utilizada tanto no desenvolvimento do *back-end* quanto em aplicações completas.

## 3.4 GIT

Segundo Ferreira (2014), o Git é um sistema de controle de versão que permite registrar e acompanhar as alterações realizadas no código-fonte ao longo do desenvolvimento.

Seu fundador, Linus Torvalds, criador do sistema operacional Linux, estava descontente com o BitKeeper, o sistema para controle de versão no desenvolvimento do kernel

Linux. O Git também é bastante utilizado em empresas em todo o mundo, inclusive no Brasil.

O uso dessa tecnologia favorece a rastreabilidade das modificações e a colaboração entre desenvolvedores.

## 3.5 GITHUB

Conforme afirma Ferreira (2014), o GitHub foi criado em 2008 como uma aplicação web para hospedagem de repositórios Git e colaboração entre desenvolvedores.

Diversos projetos de código aberto importantes são hospedados no GitHub, como jQuery, Node.js, Ruby on Rails, Jenkins, Spring, JUnit e muitos outros.

Atualmente, o GitHub se tornou uma das principais plataformas utilizadas por desenvolvedores e empresas para compartilhamento de código, controle de versões e colaboração em projetos de software.

## 3.6 BLADE (SISTEMA DE TEMPLATES)

Conforme Luciano (2016), grande parte dos frameworks web utiliza sistemas de templates para padronizar a interface visual das aplicações. Esse recurso otimiza a construção do projeto, facilitando o reuso de código e centralizando partes fixas da interface.

Com esse objetivo, o Laravel utiliza o Blade, um sistema de templates que se diferencia de outras soluções em PHP por não restringir o uso da linguagem PHP em suas páginas. Além disso, no Blade, cada view é compilada e armazenada em cache até sofrer alguma alteração, o que torna a aplicação mais eficiente e leve.

## 3.7 VISUAL STUDIO CODE

O Visual Studio Code é um editor de código-fonte desenvolvido pela Microsoft, amplamente utilizado no desenvolvimento de software. Ele oferece suporte a múltiplas linguagens de programação, integração com Git e extensões que aumentam sua funcionalidade Wikipédia (2026).

Devido à sua leveza e flexibilidade, tornou-se uma das ferramentas mais populares entre desenvolvedores web e back-end.

## 3.8 CONTROLE DE ACESSO BASEADO EM FUNÇÕES (RBAC)

O modelo de Controle de Acesso Baseado em Funções, conhecido pela sigla RBAC (*Role-Based Access Control*), é um método de autorização que gerencia o acesso de usuários a sistemas, aplicações e dados com base em funções predefinidas. Segundo IBM (s.d.), nesse modelo, o acesso não é atribuído diretamente ao indivíduo, mas ao papel que ele desempenha na organização.

Em um sistema que utiliza RBAC, um administrador atribui a cada usuário uma ou mais funções, em que cada função representa um conjunto específico de permissões ou privilégios. De acordo com IBM (s.d.), esse modelo oferece diversas vantagens:

- **Simplificação da gestão:** facilita o gerenciamento de acesso em organizações com muitos funcionários;
- **Segurança da informação:** auxilia na proteção contra ações internas indevidas e ameaças externas;
- **Princípio do menor privilégio:** garante que o usuário tenha acesso apenas aos recursos necessários ao exercício de sua função.

A aplicação do RBAC é fundamental para manter a integridade dos dados e a organização dos fluxos operacionais, permitindo que o sistema escale de forma segura à medida que novos perfis e permissões sejam necessários.

## 3.9 DOCKER

Segundo Vitalino e Castro (2018), o Docker surgiu em 2013 a partir da empresa dotCloud, fundada por Solomon Hykes. Inicialmente, a empresa atuava com soluções PaaS (*Platform as a Service*), permitindo suporte a diferentes linguagens de programação, diferentemente de outras plataformas da época.

Os autores destacam que o Docker nasceu como uma tecnologia open source baseada inicialmente em LXC (Linux Containers) e Union Filesystem, possibilitando a criação de containers leves e eficientes. Seu crescimento ocorreu rapidamente, tornando-se amplamente utilizado por empresas de grande porte, como Spotify, além de receber suporte de plataformas como AWS e Google.

De acordo com Vitalino e Castro (2018), o Docker trouxe maior facilidade na criação, distribuição e execução de aplicações em containers, garantindo mais portabilidade e consistência entre ambientes de desenvolvimento e produção. Além disso, por utilizar o mesmo kernel do sistema operacional hospedeiro, a tecnologia apresenta maior eficiência no consumo de recursos quando comparada às máquinas virtuais tradicionais.

Os autores também ressaltam que o Docker possui melhor desempenho em sistemas Linux, embora atualmente também possa ser executado em ambientes Windows e macOS. Dessa forma, o Docker tornou-se uma das principais ferramentas utilizadas em processos modernos de desenvolvimento e implantação de software.

### 3.9.1 Testes automatizados de software

O teste de software é uma atividade importante no processo de desenvolvimento, pois permite identificar falhas e avaliar se o sistema apresenta o comportamento esperado. Conforme discutido por Sousa (2023), os testes podem ser realizados com o apoio de ferramentas de automação, que permitem executar verificações de maneira repetível e analisar o funcionamento de aplicações web.

A automação de testes utiliza scripts e ferramentas para reproduzir ações e verificar resultados previamente definidos. Essa prática pode contribuir para reduzir o esforço associado à repetição manual de testes, especialmente quando o sistema passa por alterações durante o desenvolvimento. Entretanto, os resultados obtidos dependem dos cenários definidos e não garantem, isoladamente, a inexistência de defeitos na aplicação.

### 3.9.2 Testes de ponta a ponta (E2E)

Os testes de ponta a ponta, conhecidos como E2E (*End-to-End*), verificam o comportamento de uma aplicação a partir de fluxos completos de utilização. Em aplicações web, esses testes podem reproduzir ações realizadas pelo usuário no navegador e verificar se a aplicação responde conforme o esperado.

No estudo comparativo realizado por Sousa (2023), o Cypress e o Selenium são analisados como ferramentas para automatização de testes em aplicações web. O trabalho aborda características como configuração, escrita dos scripts, recursos disponíveis e execução dos testes. Essa abordagem demonstra a importância de considerar tanto as funcionalidades das ferramentas quanto as necessidades específicas do sistema que será testado.

### 3.9.3 Cypress

O Cypress é uma ferramenta voltada à automação de testes de aplicações web. Segundo Sousa (2023), a ferramenta utiliza JavaScript para a criação dos scripts de teste e foi projetada para trabalhar diretamente com a aplicação executada no navegador. Essa característica permite que os testes realizem ações na interface e verifiquem elementos e comportamentos apresentados durante a execução.

Entre as operações que podem compor um teste estão acessar uma página, localizar elementos, preencher campos, clicar em botões e verificar se o resultado esperado foi apresentado. A documentação oficial do Cypress também descreve recursos voltados à criação e execução de testes automatizados (Cypress, 2023).

Outro aspecto discutido por Sousa (2023) é o mecanismo de espera automática do Cypress. A ferramenta pode aguardar que determinados elementos ou condições estejam disponíveis antes de concluir uma verificação, reduzindo a necessidade de configurar manualmente esperas em diversos cenários. Ainda assim, os testes devem ser planejados e implementados de acordo com o comportamento esperado da aplicação.

O estudo de Sousa (2023) também aponta que a escolha de uma ferramenta de

automação deve considerar o contexto de uso. Embora o Cypress apresente facilidades na escrita e depuração dos testes, existem limitações em comparação com outras ferramentas, incluindo diferenças no suporte a linguagens, ambientes e determinados cenários de navegação. Portanto, sua adequação deve ser avaliada conforme os requisitos do projeto.

### 3.9.4 Uso do Cypress no sistema FlashFood

No sistema FlashFood, o Cypress pode ser utilizado para automatizar testes de ponta a ponta que verifiquem os fluxos implementados na aplicação. Entre os exemplos de cenários que podem ser avaliados estão a autenticação de usuários, a navegação pelas telas e as operações de gerenciamento de produtos, vendas e pedidos, desde que esses fluxos estejam implementados e façam parte dos testes desenvolvidos.

Em cada cenário, o teste deve executar as ações previstas e verificar se os resultados correspondem ao comportamento esperado. A execução repetida desses cenários após alterações no código pode auxiliar na identificação de regressões. Contudo, as conclusões sobre a qualidade do sistema devem se limitar às funcionalidades e condições efetivamente verificadas pelos testes.

## 3.10 REDIS

O Redis (*Remote Dictionary Server*) é um banco de dados NoSQL em memória amplamente utilizado como sistema de cache, armazenamento de sessões, filas de mensagens e banco de dados chave-valor. Diferentemente de outras soluções de cache, o Redis oferece suporte a diversas estruturas de dados, como listas, conjuntos, hashes e conjuntos ordenados, além de permitir persistência opcional dos dados em disco.

Segundo Redis Ltd. (2025), o armazenamento dos dados diretamente na memória RAM reduz o tempo de acesso em comparação com sistemas que dependem de operações em disco, tornando o Redis adequado a aplicações que exigem alto desempenho.

Além disso, o Redis oferece mecanismos de expiração automática de chaves (*Time To*

*Live – TTL*), permitindo que os dados armazenados sejam removidos automaticamente após determinado período, evitando inconsistências e desperdício de memória.

### 3.10.1 Cache em Aplicações Web

O cache é uma técnica utilizada para reduzir o tempo de resposta das aplicações, armazenando temporariamente informações frequentemente acessadas em uma camada de memória de alta velocidade. Dessa forma, evita-se a execução repetitiva de consultas ao banco de dados e outros processos computacionalmente custosos, contribuindo para a melhoria do desempenho e da escalabilidade dos sistemas.

Em aplicações web modernas, o uso de mecanismos de cache torna-se essencial principalmente em cenários com elevado número de acessos simultâneos. A utilização dessa estratégia reduz a carga sobre o banco de dados, diminui a latência das requisições e melhora a experiência do usuário.

### 3.10.2 Redis como Camada de Cache

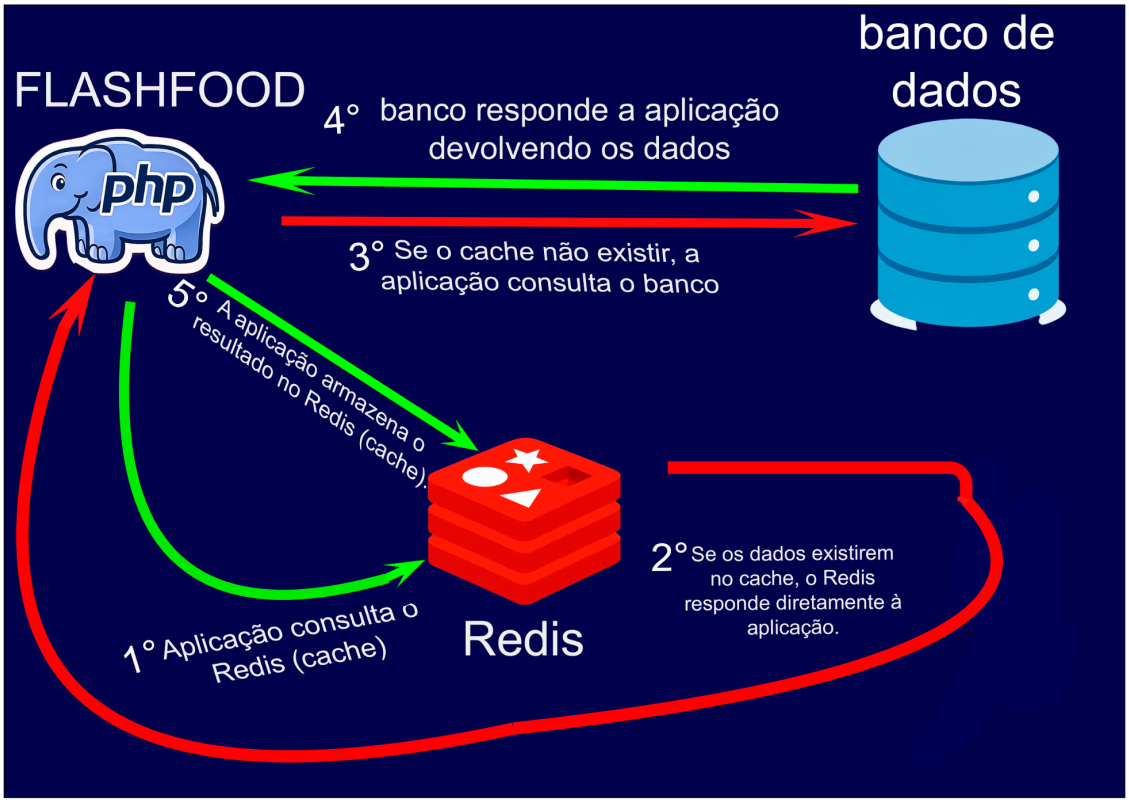
O Redis é frequentemente utilizado como uma camada intermediária entre a aplicação e o banco de dados. Quando uma informação é solicitada, a aplicação verifica inicialmente se ela está armazenada no cache. Caso esteja disponível, o dado é retornado imediatamente. Caso contrário, a informação é recuperada do banco de dados, armazenada no Redis e posteriormente enviada ao usuário.

O fluxo de consulta ao cache e ao banco de dados adotado no sistema Flashfood é apresentado na Figura 1.

Esse processo reduz significativamente a quantidade de consultas ao banco de dados, especialmente em informações acessadas repetidamente, como listas de produtos, categorias, configurações do sistema e dados estatísticos.

De acordo com Almeida *et al.* (2023), o Redis apresenta desempenho superior em operações de leitura quando comparado a bancos relacionais tradicionais, tornando-se uma alternativa para aplicações com alta demanda de acesso.

Figura 1 – Fluxo de consulta ao Redis no sistema Flashfood




Fonte: Elaborado pelo autor (2026)..

### 3.10.3 Escalabilidade Horizontal

A escalabilidade horizontal consiste na ampliação da capacidade de processamento de uma aplicação por meio da adição de novos servidores, em vez do aumento da capacidade de um único equipamento. Nesse cenário, soluções de cache como o Redis desempenham papel fundamental, reduzindo o número de acessos simultâneos ao banco de dados e distribuindo de forma mais eficiente a carga entre os servidores.

Segundo Filho (2025), arquiteturas que utilizam cache podem responder mais rapidamente às requisições dos usuários, proporcionando maior disponibilidade e desempenho.

### 3.10.4 Aplicação do Redis no Sistema Desenvolvido

Neste trabalho, o Redis foi utilizado como mecanismo de cache da aplicação desenvolvida em Laravel. Informações frequentemente consultadas, como listas de funcionários,

categorias e outros registros de baixa frequência de alteração, foram armazenadas temporariamente no Redis utilizando o mecanismo `Cache::remember()` disponibilizado pelo framework.

Quando ocorre alteração nos dados, como inclusão, edição ou exclusão de registros, o cache correspondente é invalidado por meio do método `Cache::forget()`, garantindo que as informações apresentadas aos usuários permaneçam consistentes.

Essa estratégia reduz a quantidade de consultas ao banco de dados MySQL, melhora o tempo de resposta da aplicação e aumenta sua capacidade de atender múltiplos usuários simultaneamente.

# 4 REQUISITOS DE SOFTWARE

Nesta seção são apresentados os requisitos de software definidos para o sistema, considerando as funcionalidades que devem ser disponibilizadas aos usuários e as características necessárias para seu funcionamento. Os requisitos são divididos em funcionais e não funcionais.

## 4.1 REQUISITOS FUNCIONAIS

Os requisitos funcionais de um sistema descrevem o que ele deve fazer, ou seja, os serviços e funcionalidades que o sistema deve fornecer aos usuários. Esses requisitos podem variar de descrições mais gerais até especificações detalhadas, incluindo entradas, saídas e comportamentos esperados do sistema.

Quando expressos como requisitos de usuário, são apresentados de forma mais abstrata para facilitar o entendimento. Já em nível de sistema, tornam-se mais detalhados e técnicos. A imprecisão na definição desses requisitos pode gerar interpretações diferentes pelos desenvolvedores, causando problemas, retrabalho e aumento de custos.

Em sistemas complexos, é difícil garantir que os requisitos funcionais sejam totalmente completos e consistentes, devido à grande quantidade de stakeholders e possíveis conflitos entre necessidades. Por isso, falhas ou omissões podem aparecer durante o desenvolvimento ou após a entrega do sistema (SOMMERVILLE, 2011).

Com base nas necessidades identificadas e nos objetivos definidos para o sistema, foram estabelecidos os requisitos funcionais apresentados na Tabela 2.

## 4.2 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais não estão diretamente relacionados às funções específicas do sistema, mas às características de qualidade e restrições globais, como desempenho, confiabilidade, segurança, tempo de resposta e disponibilidade. Esses requisitos podem também envolver limitações de implementação, como tipos de dis-

Tabela 2 – Requisitos funcionais do sistema

| ID | Descrição |
|---|---|
| RF01 | O sistema deve permitir o cadastro de usuários, incluindo funcionários e clientes. |
| RF02 | O sistema deve permitir que os usuários realizem autenticação e encerramento de sessão. |
| RF03 | O sistema deve controlar o acesso às funcionalidades de acordo com o perfil atribuído ao usuário. |
| RF04 | O sistema deve permitir que usuários gerenciem pedidos. |
| RF05 | O sistema deve exibir os produtos disponíveis na área do cliente. |
| RF06 | O sistema deve permitir gerenciar o carrinho de compras. |
| RF07 | O sistema deve permitir a confirmação e o registro de pedidos. |
| RF08 | O sistema deve permitir que funcionários autorizados consultem os pedidos registrados. |
| RF09 | O sistema deve permitir a localização e o acompanhamento dos pedidos em andamento. |
| RF10 | O sistema deve permitir a atualização do status do pedido para pendente, em preparo ou finalizado. |
| RF11 | O sistema deve armazenar e recuperar os dados de usuários, produtos e pedidos. |

Fonte: Fonte: Elaborado pelo autor (2026)..

positivos de entrada e saída e formas de representação de dados em interfaces com outros sistemas.

Diferentemente dos requisitos funcionais, os requisitos não funcionais são importantes para o funcionamento adequado do sistema como um todo. Falhas relacionadas à segurança, à disponibilidade ou ao desempenho podem comprometer o atendimento, a integridade dos dados e a continuidade das operações da lanchonete.

Além disso, esses requisitos são mais difíceis de serem associados a componentes específicos do sistema, pois sua implementação geralmente está distribuída em várias partes do software (SOMMERVILLE, 2011).

Os requisitos não funcionais definidos para o sistema são apresentados na Tabela 3.

Tabela 3 – Requisitos não funcionais do sistema

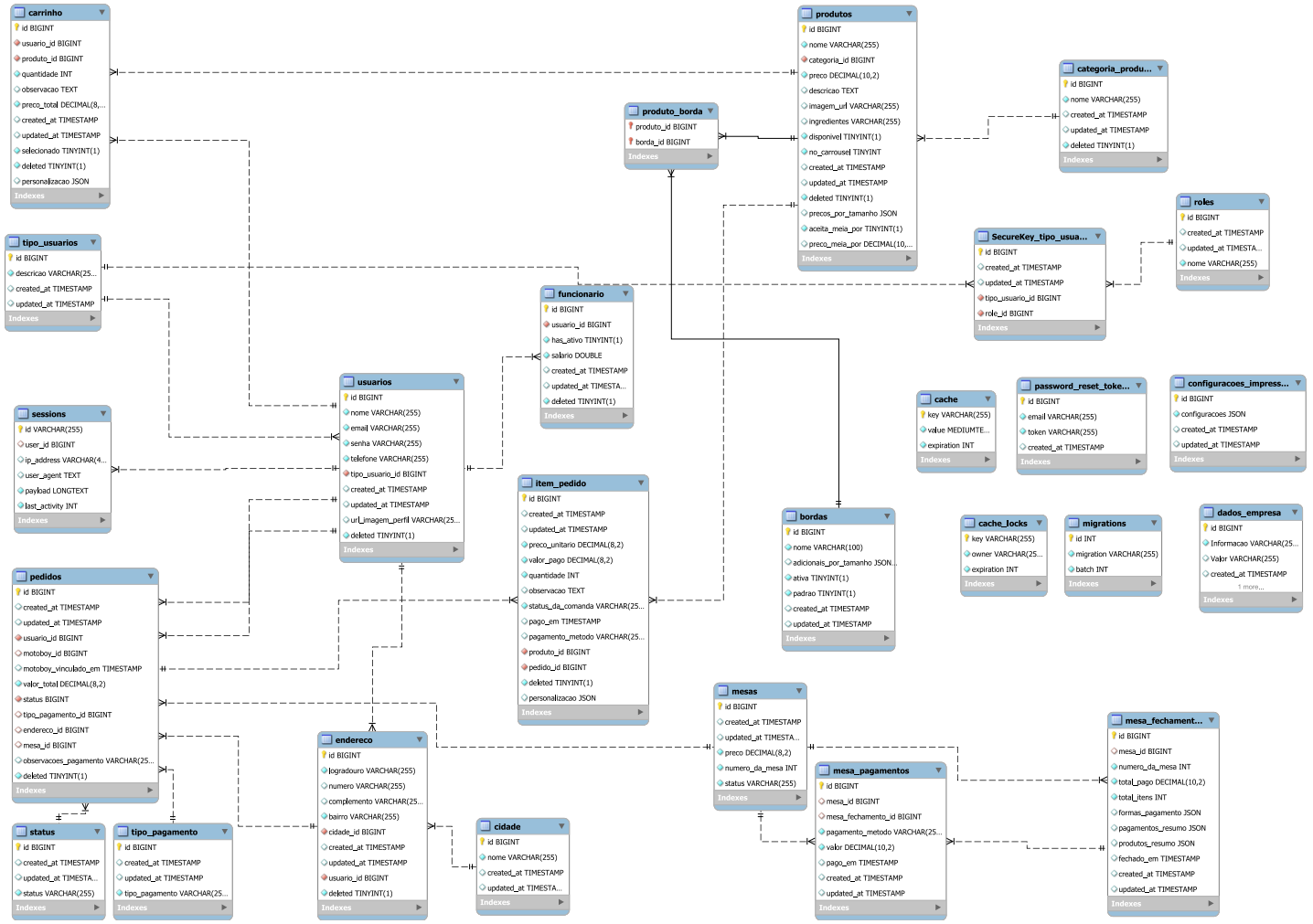
| ID | Descrição |
|---|---|
| RNF01 | O sistema deve suportar múltiplos usuários simultâneos. |
| RNF02 | As senhas devem ser armazenadas exclusivamente na forma de resumo criptográfico seguro (*hash*), conforme os mecanismos disponibilizados pelo Laravel. |
| RNF03 | O sistema deve funcionar 24 horas por dia, 7 dias por semana. |
| RNF04 | O sistema deve funcionar em navegadores como Chrome, Firefox e Edge. |
| RNF05 | O código-fonte deve seguir a organização do padrão MVC, de modo a separar apresentação, regras de negócio e acesso aos dados. |
| RNF06 | O sistema deve proteger as rotas e operações restritas por meio de autenticação e autorização baseada no perfil do usuário. |
| RNF07 | O sistema deve registrar logs de autenticação e das operações de inclusão, alteração e exclusão de registros. |

Fonte: Fonte: Elaborado pelo autor (2026)..

# 5 MODELAGEM DE SOFTWARE

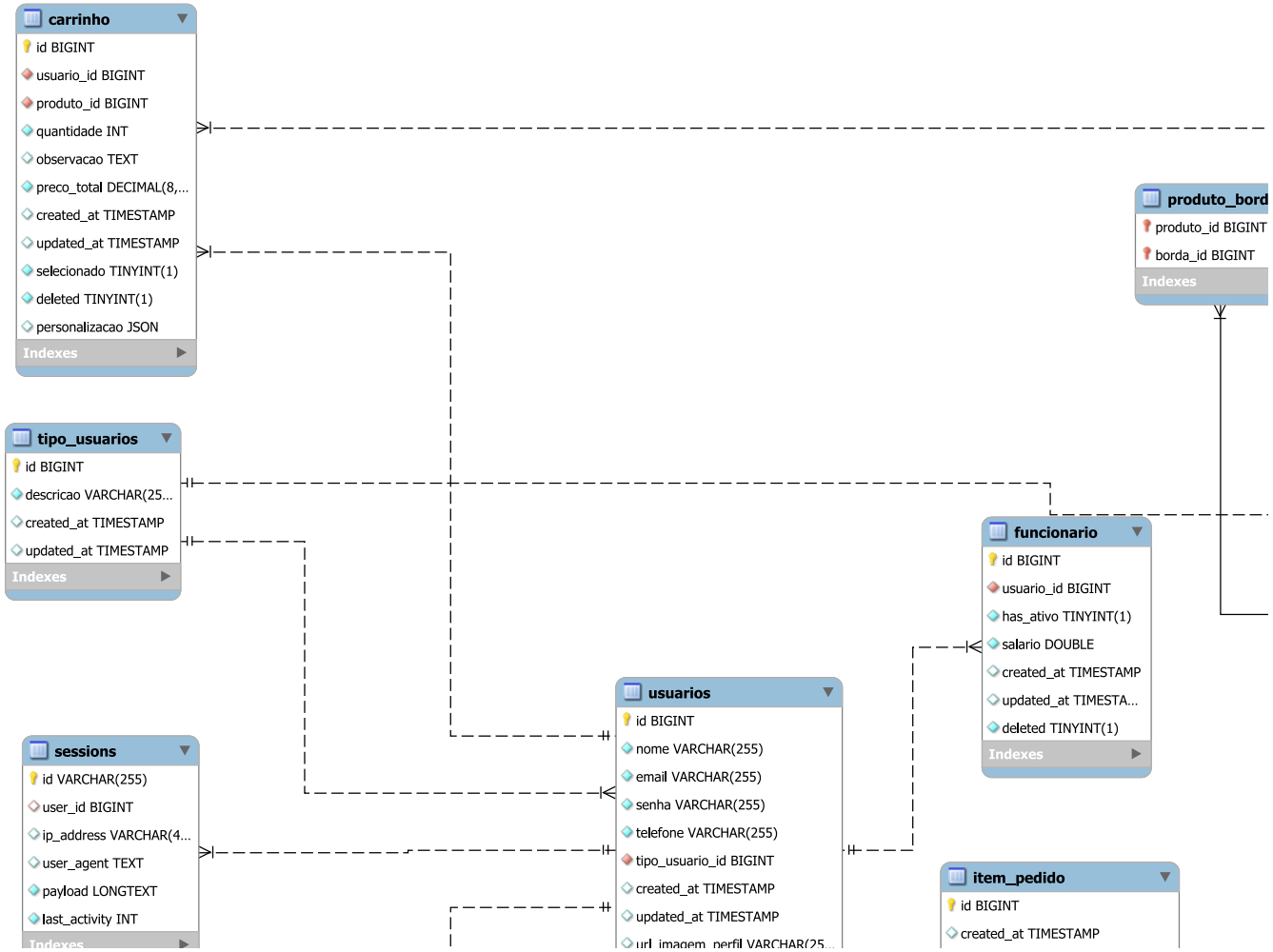
A Figura 2 apresenta a visão geral do banco de dados. Para facilitar a leitura das tabelas e de seus campos, as ampliações das diferentes regiões estão nas Figuras 3 a 6.

Figura 2 – Modelo físico do banco de dados do sistema Flashfood (visão geral)



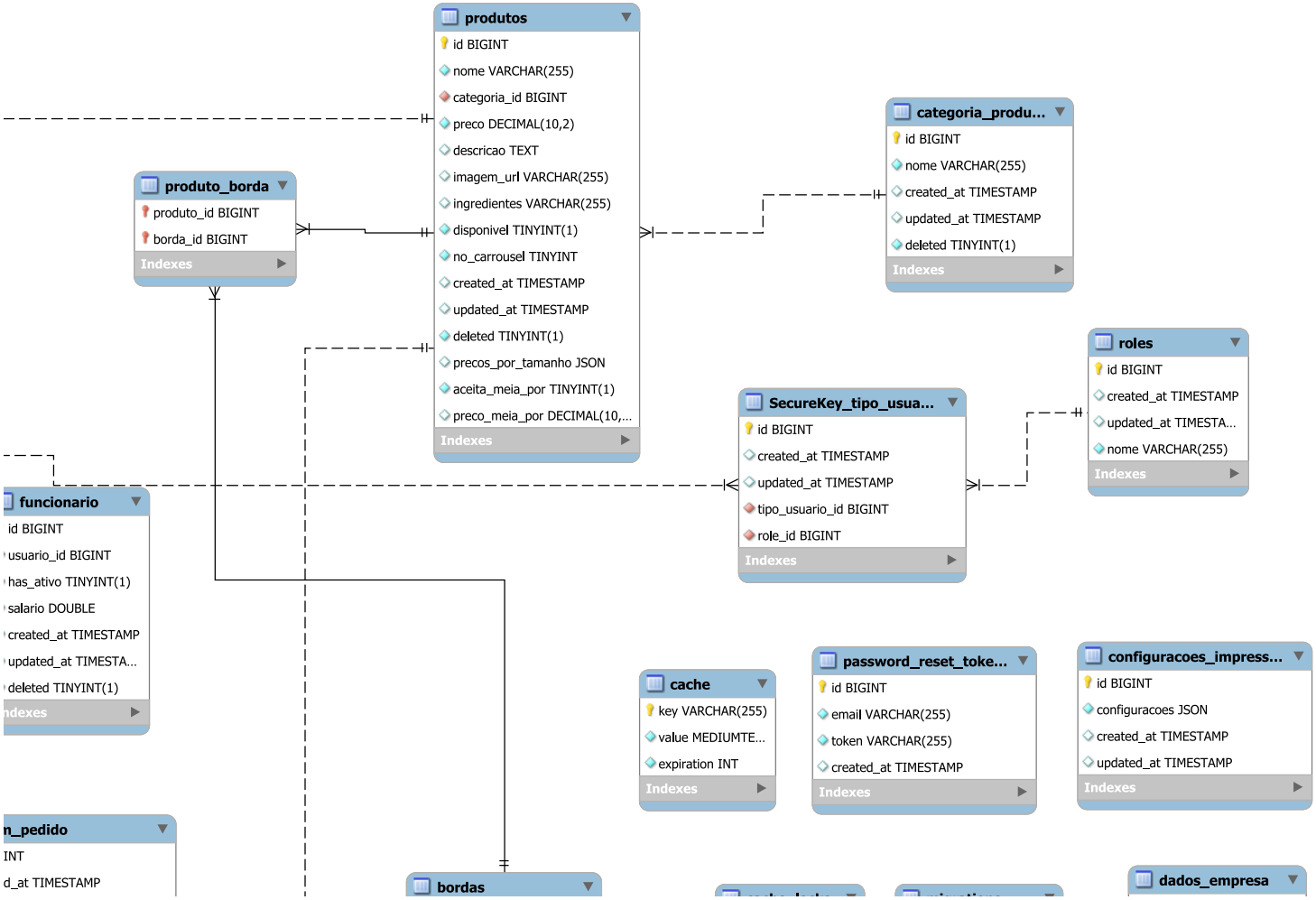

Fonte: elaborado pelo autor (2026).

Figura 3 – Detalhe do modelo físico do banco de dados – região superior esquerda



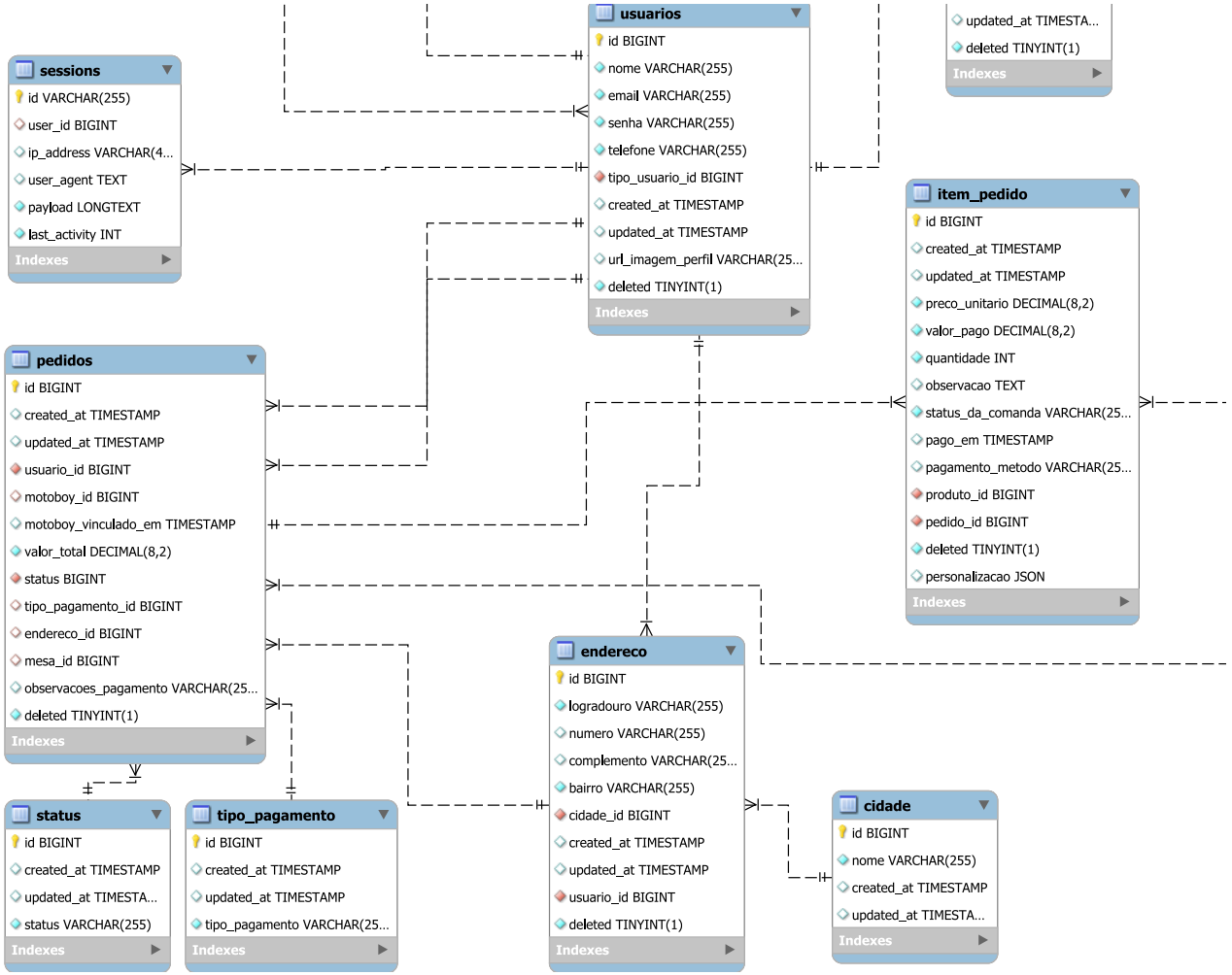

Fonte: elaborado pelo autor (2026).

Figura 4 – Detalhe do modelo físico do banco de dados – região superior direita



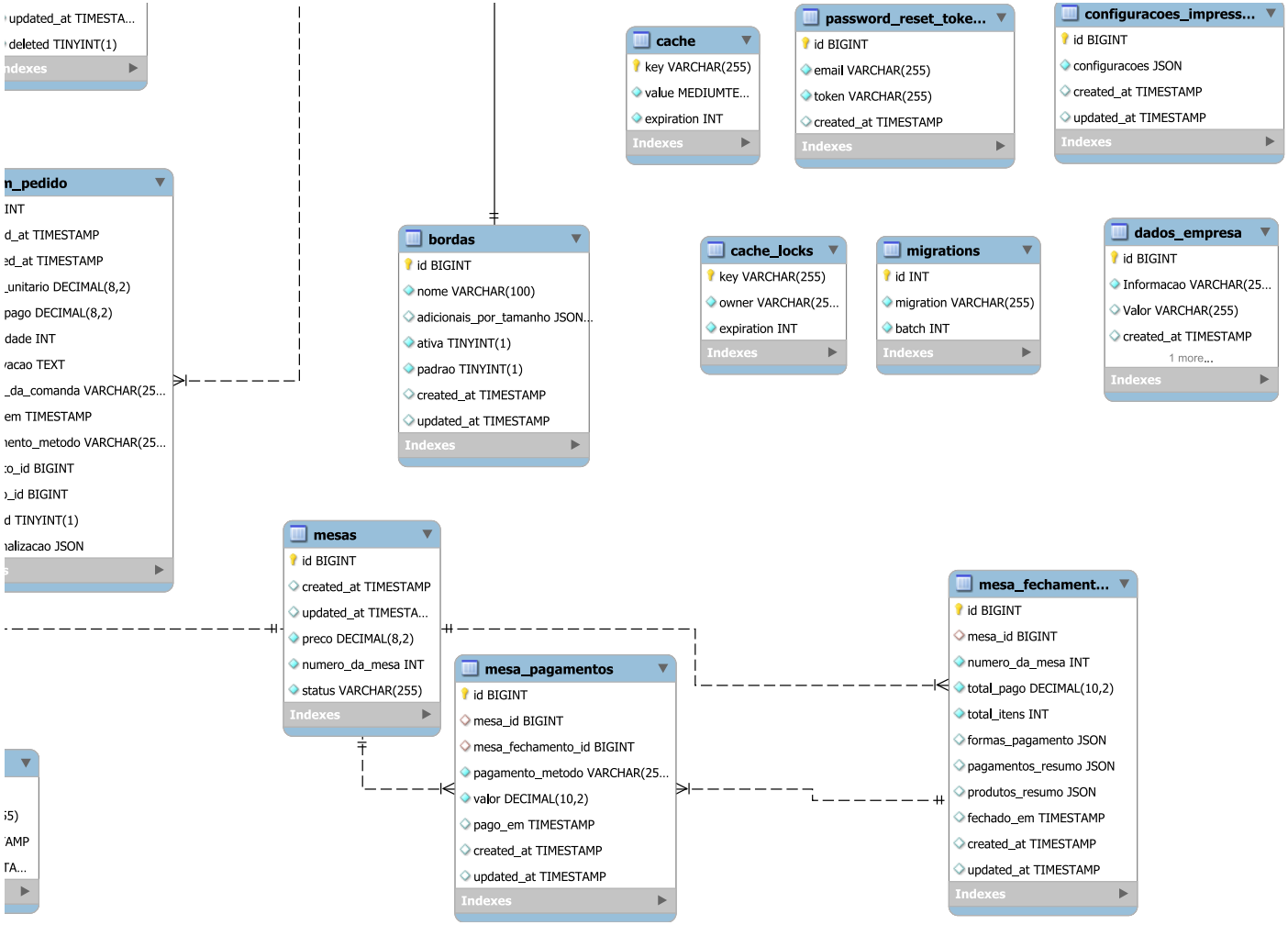

Fonte: elaborado pelo autor (2026).

Figura 5 – Detalhe do modelo físico do banco de dados – região inferior esquerda




Fonte: elaborado pelo autor (2026).

Figura 6 – Detalhe do modelo físico do banco de dados – região inferior direita




Fonte: elaborado pelo autor (2026).

# 6 TESTES E VALIDAÇÃO DO SISTEMA

## 6.1 OBJETIVO E ESCOPO DOS TESTES

Os testes de ponta a ponta foram desenvolvidos com Cypress para verificar os principais fluxos do FlashFood pela interface da aplicação. Os cenários abrangem cadastro e autenticação de usuários, controle de acesso, catálogo de produtos, carrinho de compras, pedidos, histórico de mesas e entrega. Os arquivos estão organizados no diretório `cypress/e2e/`.

## 6.2 ARQUIVOS E CENÁRIOS DE TESTE

As tabelas a seguir apresentam os arquivos Cypress e a finalidade de cada teste. A coluna de requisitos registra apenas as associações indicadas para os cenários; o símbolo – indica que não foi informado um requisito específico.

### 6.2.1 Cadastro, autenticação e controle de acesso

### 6.2.2 Produtos, cardápio, carrinho e pedidos

## 6.3 AMBIENTE E EXECUÇÃO

Os testes foram executados com Cypress, versão `15.18.0`, utilizando o navegador Google Chrome, versão `149`. O ambiente de execução utilizou Node.js, versão `24.18.0`. A aplicação foi executada em contêineres gerenciados pelo Docker Compose. O Cypress acessou o serviço `app` pelo endereço interno `http://app:8000`, configurado pela variável `CYPRESS_BASE_URL`.

## 6.4 RESULTADOS

Na execução registrada, foram executados 21 arquivos de teste, totalizando 42 casos. Todos os 42 casos foram aprovados; não houve falhas, casos pendentes ou casos ignorados. A execução completa durou 1 minuto e 46 segundos.

Tabela 4 – Testes de cadastro, autenticação e controle de acesso

| ID | Arquivo | Requisito | Aprovados | O que o teste faz |
|---|---|---|---|---|
| T01 | AcessoAdmin.cy.js | RF03 | 1 | Tenta abrir o painel administrativo sem autenticação e verifica se o sistema redireciona o visitante para a tela de login. |
| T02 | AcessoPerfil.cy.js | RF03 | 2 | Verifica que um cliente não consegue abrir o gerenciamento de produtos e que um administrador autorizado consegue acessar essa funcionalidade. |
| T03 | AcessoProtegido.cy.js | RF03 | 8 | Acessa oito páginas protegidas, incluindo perfil, pedidos, carrinho, mesas e entregas, sem uma sessão ativa e verifica o redirecionamento para o login. |
| T04 | Cadastro.cy.js | RF01 | 1 | Cadastra um cliente pela página de registro e, em seguida, confirma que ele consegue entrar no sistema. |
| T05 | CriarFuncionario.cy.js | RF01 | 1 | Verifica o cadastro de um funcionário por meio do painel administrativo. |
| T06 | DeletarFuncionario .cy.js | – | 1 | Verifica a remoção de um funcionário pelo painel administrativo. |
| T07 | LogandoComFuncionario Criado.cy.js | RF01, RF02 | 2 | Cadastra um funcionário autorizado e verifica se ele consegue se autenticar no sistema. |
| T08 | Logout.cy.js | RF02 | 1 | Encerra a sessão pelo menu e verifica se uma página protegida volta a exigir autenticação. |
| T09 | ValidacaoFormularios .cy.js | – | 4 | Verifica campos obrigatórios nas telas de login, cadastro e redefinição de senha, além do link para voltar ao login. |
| T10 | loginAdmin.cy.js | RF02 | 1 | Autentica um administrador e verifica se o painel administrativo é aberto. |
| T11 | LoginInvalido.cy.js | RF02 | 1 | Tenta entrar com credenciais inválidas e verifica se o sistema permanece na tela de login. |

Fonte: Elaborado pelo autor (2026)..

Tabela 5 – Testes de produtos, cardápio, carrinho e pedidos

| ID | Arquivo | Requisito | Aprovados | O que o teste faz |
|---|---|---|---|---|
| T12 | `AtualizarProduto.cy.js` | – | 1 | Abre a edição de um produto no painel administrativo, altera seus dados e verifica se a atualização foi realizada. |
| T13 | `BuscaProduto.cy.js` | – | 1 | Pesquisa produtos pelo nome na tabela administrativa e verifica se os resultados são filtrados. |
| T14 | `Carrinho.cy.js` | RF06 | 1 | Adiciona um produto ao carrinho, altera a quantidade, confere o subtotal e remove o produto. |
| T15 | `CatalogoProdutos.cy.js` | RF05 | 1 | Cadastra um produto disponível e verifica se ele aparece no catálogo de bebidas do cliente com preço e descrição. |
| T16 | `CriandoProduto.cy.js` | RF11 | 1 | Cria um produto pelo painel administrativo e verifica se seus dados aparecem na listagem. |
| T17 | `DeletandoProduto.cy.js` | – | 1 | Verifica a remoção de um produto pelo painel administrativo. |
| T18 | `NavegacaoCardapio .cy.js` | RF05 | 7 | Verifica as páginas do cardápio, a navegação até a seção de bebidas e a exibição do controle do menu em uma tela pequena. |
| T19 | `PersonalizacaoPizza .cy.js` | – | 3 | Verifica o formulário de preços por tamanho, o salvamento e a edição desses preços e a disponibilidade das bordas ativas para pizzas. |
| T20 | `FluxoCompraEntrega .cy.js` | RF04, RF07–RF11 | 1 | Percorre o fluxo de compra: registra o cliente e o pedido, confere os dados salvos, consulta e atualiza o pedido no painel administrativo, acompanha o status como cliente e verifica as etapas da entrega. |
| T21 | `MesasHistorico.cy.js` | – | 2 | Verifica os resumos e filtros do histórico de mesas e confirma a aplicação do filtro por datas. |

Fonte: Elaborado pelo autor (2026)..

## 6.5 EVIDÊNCIAS

Como evidência da execução, foi considerado o relatório de resultados do Cypress enviado pelo autor, que registra 21 arquivos e 42 casos aprovados, sem falhas, pendências ou casos ignorados. Essa execução não gerou capturas de tela nem vídeo.

# 7 CONCLUSÃO

Este trabalho apresentou a proposta e o desenvolvimento de uma aplicação web para apoiar a gestão operacional de uma lanchonete. A solução foi concebida para centralizar informações de usuários, produtos e pedidos, reduzindo a dependência de registros manuais e favorecendo a organização das rotinas do estabelecimento.

A adoção do Laravel e do padrão MVC contribuiu para separar as responsabilidades da aplicação. O MySQL foi utilizado para a persistência dos dados, enquanto o Redis foi incorporado como camada de cache para informações consultadas com frequência. O controle de acesso baseado em funções permitiu relacionar as permissões às atribuições dos diferentes usuários. Docker, Git e GitHub complementaram o ambiente de desenvolvimento ao apoiar, respectivamente, a padronização da execução e o controle de versões.

Os requisitos levantados abrangem o cadastro e a autenticação de usuários, o gerenciamento e a exibição de produtos, o carrinho de compras e o acompanhamento dos pedidos. Esses recursos atendem às necessidades centrais identificadas no contexto analisado. Entretanto, a comprovação dos ganhos de desempenho, usabilidade e eficiência operacional depende da execução e da documentação de testes em ambiente representativo da lanchonete.

## 7.1 TRABALHOS FUTUROS

Como trabalhos futuros, propõe-se implementar a autenticação por meio de contas Google. Essa funcionalidade permitiria que o usuário acessasse o sistema usando uma conta existente, sem precisar criar uma senha específica para a aplicação. A implementação deverá associar a conta Google ao perfil correspondente no sistema e manter as permissões de acesso definidas para cada tipo de usuário.

Também se propõe integrar o sistema ao Google Maps para auxiliar no planejamento das entregas. A partir dos endereços cadastrados nos pedidos, a aplicação poderia calcular automaticamente rotas, estimar distâncias e tempos de deslocamento e, quando houver várias entregas, sugerir uma sequência de atendimento. A viabilidade dessa

integração deverá considerar a validação dos endereços, a proteção dos dados dos clientes e os custos de uso do serviço.

# REFERÊNCIAS


ALMEIDA, D. *et al.* Performance comparison of redis, memcached, mysql, and postgresql: A study on key-value and relational databases. In: **Second International Conference on Smart Technologies for Smart Nation**. [S.l.: s.n.], 2023. p. 902–907.

Cypress. **Why Cypress?** 2023. Disponível em: <https://docs.cypress.io/guides/overview/why-cypress>. Acesso em: 17 out. 2026.

DALL'OGLIO, P. **PHP: programando com orientação a objetos**. São Paulo: Novatec Editora, 2015. 552 p.

FERREIRA, R. **Controlando versões com Git e GitHub**. [S.l.]: Casa do Código, 2014. 199 p.

FILHO, N. S. Comparative analysis between monolithic architecture and microservices in .net applications. **Leaders Tec**, v. 2, n. 13, 2025.

HIDAYAT, T.; FARELL, G. R. Rancang bangun sistem informasi point of sale di kedai kopi. **Jurnal Pendidikan Tambusai**, v. 7, n. 2, p. 18372–18381, 2023. Disponível em: <https://jptam.org/index.php/jptam/article/view/9273>.

IBM. **O que é controle de acesso baseado em função (RBAC)?** s.d. Acesso em: 30 abr. 2026. Disponível em: <https://www.ibm.com/>.

JINGGA, F.; LIMANTARA, N. Pembangunan model restaurant management system. **ComTech: Computer, Mathematics and Engineering Applications**, v. 5, n. 2, p. 973–982, 2014. Disponível em: <https://journal.binus.ac.id/index.php/comtech/article/view/2348>.

KHAIRUNNISA, G. W.; ARWANI, I.; HANGGARA, B. T. Pengembangan sistem informasi point of sales berbasis web menggunakan framework laravel: Studi kasus meetup station. **Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer**, v. 6, n. 4, p. 1858–1864, 2022. Disponível em: <https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/10937>.

LATHIEF, F. M.; HASDIANA; LUBIS, H. Perancangan sistema point of sales multi payment berbasis web menggunakan framework laravel pada umkm mr. boba. **Jurnal Ilmu Komputer dan Sistem Informasi**, v. 2, n. 3, 2023. Disponível em: <https://jurnal.unity-academy.sch.id/index.php/jirsi/article/view/72>.

LUCIANO. **Blade Engine: Utilizando templates no Laravel**. 2016. Acesso em: 29 abr. 2026. Disponível em: <https://www.devmedia.com.br/blade-engine-utilizando-templates-no-laravel/36749>.

MILANI, A. **MySQL: guia do programador**. São Paulo: Novatec Editora, 2007. 400 p.

N.RICARDO. **O que é Laravel: principais recursos, onde usar e vantagens do framework PHP**. 2025. Acesso em: 29 abr. 2026. Disponível em: <https://www.hostinger.com/br/tutoriais/o-que-e-laravel>.

Redis Ltd. **Redis Documentation**. 2025. Disponível em: <https://redis.io/documentation>.

RUSMARA, R. H. Pengembangan sistem kasir berbasis web di natta coffee: Studi implementasi framework laravel. **Jurnal Informatika dan Teknik Elektro Terapan**, v. 13, n. 2, 2025. Disponível em: <https://journal.eng.unila.ac.id/index.php/jitet/article/view/6414>.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Revisão técnica de Kechi Hirama.

SOUSA, R. E. d. **Um estudo comparativo entre ferramentas de automação de testes: Selenium e Cypress**. Monografia (Trabalho de Conclusão de Curso) — Universidade Federal do Ceará, Sobral, 2023. Disponível em: <https://repositorio.ufc.br/bitstream/riufc/76304/1/2023_tcc_resousa.pdf>. Acesso em: 17 out. 2026.

VITALINO, J. F. N.; CASTRO, M. A. N. **Descomplicando o Docker**. 2. ed. Rio de Janeiro: Brasport, 2018. ISBN 978-85-7452-902-8.

Wikipédia. **Visual Studio Code**. 2026. Acesso em: 29 abr. 2026. Disponível em: <https://pt.wikipedia.org/wiki/Visual_Studio_Code>.